جزوه جامع و بی‌نظیر فصل دوم (بخش ۴ تا ۶: فرآیند تولد یک برنامه)

این جزوه بر اساس بخش‌های چهارم تا ششم از فصل دوم فایل ارسالی شما و با هدف پوشش **کامل، عمیق و مفهومی** تمام مطالب، تعاریف فنی، فرآیندها و مثال‌های ذکر شده تهیه شده است. هدف، درک کامل مفاهیم برای کسب بالاترین نمره است.

۲-۴: برنامه‌های سیستمی (System Programs)

برنامه‌های سیستمی، که به آن‌ها **ابزارهای سودمند سیستمی (System Utilities)** نیز گفته می‌شود، یک لایه نرم‌افزاری هستند که بین سیستم‌عامل و برنامه‌های کاربردی قرار می‌گیرند. آن‌ها محیطی راحت‌تر و قدرتمندتر برای توسعه و اجرای برنامه‌ها فراهم می‌کنند.

- **تفاوت با برنامه‌های کاربردی:** خط مرز مشخصی وجود ندارد، اما برنامه‌های سیستمی معمولاً با خود سیستم‌عامل عرضه می‌شوند و برای **مدیریت سیستم** به کار می‌روند (مانند ابزارهای فرمت کردن دیسک)، در حالی که برنامه‌های کاربردی برای **حل مسائل کاربران نهایی** طراحی شده‌اند (مانند یک واژه‌پرداز).
- **تفاوت با فراخوانی‌های سیستمی:** بسیاری از برنامه‌های سیستمی، در واقع **پوشش‌های کاربرپسند (User-friendly Wrappers)** برای فراخوانی‌های سیستمی هستند. برای مثال، وقتی شما در خط فرمان دستور `mkdir my_folder` را برای ایجاد یک دایرکتوری جدید تایپ می‌کنید، برنامه سیستمی `mkdir` در پشت صحنه، فراخوانی سیستمی `mkdir()` را با پارامتر مناسب صدا می‌زند.

دسته‌بندی برنامه‌های سیستمی:

1. **مدیریت فایل:** شامل تمام ابزارهایی که برای کار با فایل‌ها و دایرکتوری‌ها نیاز داریم: ایجاد، حذف، کپی، تغییر نام، چاپ و لیست کردن (`ls`, `cp`, `rm`, `mv` در یونیکس).
2. **اطلاعات وضعیت (Status Information):** ابزارهایی که اطلاعات دقیقی در مورد وضعیت فعلی سیستم ارائه می‌دهند. این اطلاعات می‌تواند شامل تاریخ و ساعت، میزان حافظه یا فضای دیسک آزاد، لیست کاربران وارد شده، یا اطلاعات پیچیده‌تر برای تحلیل کارایی و عیب‌یابی باشد (`ps`, `top`, `df` در یونیکس).
3. **ویرایش فایل:** ویرایشگرهای متنی (مانند `vi`, `nano` در لینوکس یا `Notepad` در ویندوز) که برای ایجاد و تغییر محتوای فایل‌ها به کار می‌روند.
4. **پشتیبانی از زبان‌های برنامه‌نویسی:** مجموعه‌ای از ابزارها که برای توسعه نرم‌افزار ضروری هستند: **کامپایلرها** (مانند `gcc`)، **اسمبلرها، دیباگرها** (مانند `gdb`) و **مفسرها** (مانند Python یا Ruby).
5. **بارگذاری و اجرای برنامه (Loading and Executing):** شامل **بارگذارها (Loaders)** و **پیونددهنده‌ها (Linkers)** که مسئول آماده‌سازی یک برنامه کامپایل شده برای اجرا در حافظه هستند (در بخش بعد به تفصیل بررسی می‌شوند).
6. **ارتباطات (Communications):** برنامه‌هایی که امکان برقراری ارتباط بین کاربران و سیستم‌های مختلف را فراهم می‌کنند، مانند کلاینت‌های ایمیل، مرورگرهای وب، و ابزارهای انتقال فایل (`ssh`, `ftp`, `ping`).
7. **سرویس‌های پس‌زمینه (Background Services):** برنامه‌هایی که از زمان راه‌اندازی سیستم به طور مداوم در پس‌زمینه اجرا می‌شوند تا وظایف مشخصی را انجام دهند.
    - در یونیکس/لینوکس به آن‌ها **دیمن (Daemon)** گفته می‌شود.
    - در ویندوز به آن‌ها **سرویس (Service)** گفته می‌شود.

- **مثال‌ها:** یک دیمن شبکه که به درخواست‌های ورودی گوش می‌دهد، یک زمان‌بند که کارها را در زمان‌های مشخص اجرا می‌کند، یا یک سرویس ناظر بر خطاهای سیستم.

۵-۲: پیونددهنده‌ها و بارگذارها (Linkers and Loaders)

یک برنامه از زمان نوشته شدن به زبان برنامه‌نویسی تا اجرا شدن در حافظه، سفری چند مرحله‌ای را طی می‌کند. دو بازیگر اصلی در این سفر، **پیونددهنده** و **بارگذار** هستند.

1. **کد منبع (Source Code):** برنامه‌ای که توسط برنامه‌نویس نوشته می‌شود (مثلاً `main.c`).
2. **کامپایلر (Compiler):** کد منبع را به **کد ماشین** ترجمه کرده و یک **فایل مقصد (Object File)** ایجاد می‌کند (مثلاً `main.o`). این فایل هنوز قابل اجرا نیست و آدرس‌های آن نهایی نشده‌اند (به آن **فایل مقصد جابجاشدنی (Relocatable Object File)** می‌گویند).
3. **پیونددهنده (Linker):**
   - **وظیفه:** چندین فایل مقصد و کتابخانه‌های لازم (مانند کتابخانه استاندارد C یا کتابخانه ریاضی `libm`) را با هم ترکیب کرده و یک **فایل اجرایی (Executable File)** واحد می‌سازد (مثلاً `main` در لینوکس یا `main.exe` در ویندوز).
   - **تجدید مکان (Relocation):** در این مرحله، پیونددهنده آدرس‌های نسبی در فایل‌های مقصد را به آدرس‌های نهایی در فایل اجرایی تبدیل می‌کند.
1. **بارگذار (Loader):**
   - **وظیفه:** بارگذار، یک برنامه سیستمی است که فایل اجرایی را از دیسک خوانده و آن را در حافظه اصلی (RAM) بارگذاری می‌کند تا برای اجرا آماده شود.
   - **فرآیند اجرا در یونیکس:**
     1. وقتی کاربر نام برنامه (`./main`) را در پوسته تایپ می‌کند، پوسته ابتدا با فراخوانی سیستمی `fork()` یک پردازه جدید ایجاد می‌کند.
     2. سپس، در پردازه جدید، با فراخوانی سیستمی `exec()`، بارگذار را احضار می‌کند.
     3. بارگذار، کد و داده‌های برنامه `main` را در فضای آدرس پردازه جدید بارگذاری کرده و اجرا را از **نقطه ورود (Entry Point)** برنامه آغاز می‌کند.

- **کتابخانه‌های اتصال پویا (Dynamically Linked Libraries - DLLs):**
  - **مشکل اتصال ایستا:** در روش سنتی (اتصال ایستا)، کد تمام کتابخانه‌ها در خود فایل اجرایی کپی می‌شود که باعث بزرگ شدن حجم فایل و هدر رفتن حافظه (در صورت استفاده همزمان چندین برنامه از یک کتابخانه) می‌شود.
  - **راه حل اتصال پویا:** در این روش، کد کتابخانه‌ها در فایل اجرایی کپی **نمی‌شود**. به جای آن، پیونددهنده فقط یک ارجاع به کتابخانه در فایل اجرایی قرار می‌دهد. در **زمان اجرا (Run-time)**، زمانی که برنامه برای اولین بار به تابع کتابخانه‌ای نیاز پیدا می‌کند، آن کتابخانه به صورت پویا در حافظه بارگذاری شده و به برنامه متصل می‌شود.
  - **مزایا:**
    - **صرفه‌جویی در فضای دیسک و حافظه:** چندین برنامه می‌توانند از یک نسخه مشترک از کتابخانه در حافظه استفاده کنند.
    - **به‌روزرسانی آسان:** برای به‌روزرسانی یک کتابخانه، کافی است فایل کتابخانه جایگزین شود و نیازی به کامپایل مجدد تمام برنامه‌هایی که از آن استفاده می‌کنند، نیست.

۲-۶: چرا برنامه‌ها خاص سیستم‌عامل هستند؟

چرا یک برنامه کامپایل شده روی ویندوز، روی لینوکس اجرا نمی‌شود، حتی اگر هر دو روی یک پردازنده یکسان (مثلاً اینتل) باشند؟ این عدم سازگاری به دلایل متعددی در سطوح مختلف سیستم به وجود می‌آید.

سه روش برای دستیابی به قابلیت حمل برنامه (Portability):

1. **استفاده** از زبان‌های مفسری (Interpreted Languages):
   - **روش کار:** برنامه‌هایی که به زبان‌هایی مانند Python یا Ruby نوشته شده‌اند، به کد ماشین کامپایل نمی‌شوند. در عوض، یک برنامه **مفسر (Interpreter)** خط به خط کد منبع را خوانده و آن را در لحظه به دستورات قابل فهم برای سیستم‌عامل و سخت‌افزار میزبان ترجمه می‌کند.
   - **مزیت:** تا زمانی که یک مفسر برای یک سیستم‌عامل موجود باشد، برنامه روی آن اجرا خواهد شد.
   - **عیب:** کارایی پایین‌تر نسبت به کد کامپایل شده (بومی).
1. **استفاده از ماشین مجازی (Virtual Machine):**
   - **روش کار:** زبان‌هایی مانند جاوا، کد منبع را به یک کد میانی به نام **بایت‌کد (Bytecode)** کامپایل می‌کنند. این بایت‌کد روی یک **ماشین مجازی جاوا (JVM)** اجرا می‌شود که بخشی از **محیط زمان اجرا (RTE)** است.
   - **مزیت:** شعار "یک بار بنویس، همه‌جا اجرا کن". تا زمانی که JVM برای یک پلتفرم موجود باشد، برنامه جاوا روی آن اجرا می‌شود.
   - **عیب:** کارایی ممکن است کمی پایین‌تر از کد بومی باشد و سربار خود ماشین مجازی نیز وجود دارد.
1. **استفاده از API استاندارد و کامپایل مجدد:**
   - **روش کار:** برنامه‌نویس با استفاده از یک API استاندارد (مانند **POSIX** برای یونیکس) کد را می‌نویسد. سپس برای هر سیستم‌عامل هدف، کد منبع باید به طور جداگانه **کامپایل مجدد (Recompile)** شود تا فایل اجرایی بومی آن سیستم تولید گردد.
   - **مزیت:** بالاترین کارایی ممکن.
   - **عیب:** فرآیند پورت کردن و کامپایل مجدد می‌تواند زمان‌بر و پیچیده باشد.

چالش‌های عمیق‌تر سازگاری:

- **قالب فایل اجرایی (Executable Format):** هر سیستم‌عامل فرمت خاصی برای فایل‌های اجرایی خود دارد که ساختار فایل و نحوه قرارگیری کد و داده‌ها را مشخص می‌کند (مثلاً **ELF** در لینوکس، **PE** در ویندوز، و **Mach-O** در macOS).
- **مجموعه دستورالعمل‌های پردازنده (CPU Instruction Set):** برنامه‌ها باید برای معماری پردازنده خاصی (مانند 86x یا ARM) کامپایل شوند.
- **تفاوت** در فراخوانی‌های **سیستمی:** هر سیستم‌عامل مجموعه فراخوانی‌های سیستمی منحصربه‌فرد خود را دارد.
- **رابط دودویی برنامه (Application Binary Interface - ABI):**
  - **تعریف:** ABI یک استاندارد سطح پایین است که جزئیات دقیقی مانند نحوه ارسال پارامترها به توابع، سازماندهی پشته، و اندازه انواع داده‌ها را برای یک معماری و سیستم‌عامل خاص تعریف می‌کند.
  - **اهمیت:** حتی اگر دو سیستم‌عامل از یک پردازنده و فرمت اجرایی یکسان استفاده کنند، اگر